

Local Automation Options for Deploying Business Central AL App Files Without Pipeline Changes

Executive summary

If you cannot modify Azure DevOps pipelines, the strongest local deployment pattern from a Windows developer PC is to separate **build** from **deployment** and automate deployment with either **official Business Central APIs** or **official on-premises PowerShell cmdlets**. For **Business Central SaaS**, the highest-confidence general-purpose path today is the **Automation API** (`/api/microsoft/automation/v2.0`) for uploading and installing per-tenant extensions, because Microsoft documents the exact `extensionUpload -> extensionContent -> Microsoft.NAV.upload -> extensionDeploymentStatus` flow, including scheduling and schema sync options. For **Business Central on-premises**, the most reliable path is still the **Business Central Administration Shell** cmdlets such as `Publish-NAVApp`, `Sync-NAVApp`, `Install-NAVApp`, and `Start-NAVAppDataUpgrade`, optionally invoked remotely from the developer PC by PowerShell remoting. ¹

For code you can copy or adapt, the most useful GitHub sources are **Microsoft's AL-Go** deployment scripts and **BcContainerHelper**. AL-Go's deployment action explicitly uses `Publish-PerTenantExtensionApps` for production-style SaaS deployment through the Automation API and `Publish-BcContainerApp` for development-endpoint publishing in sandbox scenarios. BcContainerHelper already wraps authentication, dependency sorting, upload, polling, schema sync, and optional unpublish of older versions, which makes it the fastest route to a working local PowerShell solution. The important caveat is that Microsoft has announced that official support for **BcContainerHelper in AL-Go ends on October 1, 2027**, so it is a strong adaptation source now, but not the best long-term strategic dependency. ²

The **Business Central Admin Center API** is excellent for **environment selection, app inventory, install/update/uninstall operations, and surrounding administrative automation**. Microsoft's current public docs clearly cover installed apps, available updates, install, update, uninstall, and operations tracking. Microsoft's 2026 release-plan material also says that full PTE upload/install/manage support is becoming available directly in the admin center and its API, which would reduce the need to go through the Automation API for SaaS PTE uploads. However, in the public operational documentation reviewed here, the fully documented, copy-paste-ready upload flow is still the **Automation API**, so that remains the safest recommendation today. ³

Several alternatives are **not** good primary deployment surfaces. Standard **SOAP, OData, and REST data APIs** are for exposing Business Central data and application logic, not for uploading `.app` packages; SOAP is also deprecated. **VS Code / AL Language publish paths** remain useful for interactive sandbox development but are less suitable for unattended PowerShell deployment from a developer PC, and DEV extensions are sandbox-only and can be removed on sandbox update or relocation. **App Management API** is relevant for ISVs managing Business Central service-side app artifacts at scale, but Microsoft states that direct API access is currently only for **Embed** app scenarios, not general add-on/connect partner deployment. ⁴

Reusable Microsoft and GitHub building blocks

The most reusable Microsoft-maintained GitHub asset is **AL-Go**. Microsoft describes AL-Go for GitHub as the standard set of templates and actions for Business Central AL DevOps, and the repo's deployment script shows the two key branches of logic that matter for your scenario: **development endpoint publishing** via `Publish-BcContainerApp -useDevEndpoint` and **production-style SaaS deployment** via `Publish-PerTenantExtensionApps` against the Automation API. That makes AL-Go a very good source to **copy deployment logic into a local PowerShell module**, even if you cannot use GitHub Actions or Azure DevOps pipelines directly. ⁵

The most immediately practical PowerShell source is **BcContainerHelper**. Its `Publish-PerTenantExtensionApps.ps1` function already accepts `clientId`, `clientSecret`, `tenantId`, `environment`, and an array of `.app` files; it sorts apps by dependency, creates the `extensionUpload` record, uploads the binary stream to `extensionContent`, calls `Microsoft.NAV.upload`, polls `extensionDeploymentStatus`, and can unpublish previous uninstalled versions on sufficiently new service versions. If you want the shortest path from "research" to "working local script," this is the most copyable artifact in the ecosystem. ⁶

For admin-center-level automation, Microsoft's **BCTech** repository is a good secondary source. It includes PowerShell samples for the Admin Center API, including environment and app-management examples. Those samples are especially useful when you want to validate environment names, inspect installed apps, or automate post-deployment actions around updates and status tracking without touching CI/CD definitions. ⁷

Community projects are useful, but they should be treated as accelerators rather than canonical deployment surfaces. **ALOps** is Azure DevOps-oriented, and **ALOps-External-Deployer** adds an external deployment service for on-premises environments. These are adaptable, but they introduce more moving parts and less desirable security posture than a straightforward official API or official admin-shell approach. Likewise, "BcAdmin" and similar community utilities exist, but they are not official Microsoft deployment surfaces and are best viewed as wrappers around the official APIs rather than strategic foundations. ⁸

API feasibility, authentication, permissions, and security

For **SaaS PTE upload/install**, Microsoft's **Automation API** is fully feasible and officially documented. The documented flow is: create `extensionUpload`, upload the app binary to `extensionContent`, call `Microsoft.NAV.upload`, then poll `extensionDeploymentStatus`. Microsoft also documents that installation through this mechanism is **SaaS-only** for per-tenant extensions. The request supports `"schedule"` values of `"Current version"`, `"Next minor version"`, or `"Next major version"`, plus `"schemaSyncMode"` values of `"Add"` or `"Force Sync"`. ⁹

For **authentication to Automation APIs and Business Central APIs**, the recommended non-interactive model is a Microsoft Entra application using **client credentials**. Microsoft's S2S guidance says that `Automation.ReadWrite.All` is required for `/api/microsoft/automation` endpoints and `API.ReadWrite.All` is required for Business Central APIs and web services. The app must then also be set up inside Business Central and granted object/permission-set access. Microsoft explicitly notes that

applications **cannot** be assigned `SUPER`, and points to `D365 AUTOMATION` and `EXTEN. MGT. - ADMIN` as the typical permission sets for automation and extension management. ¹⁰

For the **Admin Center API**, the authentication model is also Microsoft Entra app-based OAuth, with `AdminCenter.ReadWrite.All`. The app must be authorized in the Business Central Admin Center, and Microsoft notes that some app-management operations also require permissions assigned **inside the environment**. Microsoft's app-management page further states that `install` and `uninstall` require the `EXTEN. MGT. - ADMIN` permission set on the Business Central user or authorized Entra application. For delegated administration, Microsoft says the app must be added to the **AdminAgents** group. ¹¹

For **on-premises**, the main official route is the Business Central Administration Shell. Microsoft states that the shell is installed with server components, supports both PowerShell 7 and Windows PowerShell 5 compatibility layers in modern versions, and supports **PowerShell remoting** so you can run admin cmdlets on the remote BC server from a local developer PC. This means "local deployment from the dev PC" is entirely realistic for on-prem as long as you can reach the server and are allowed to remote in. ¹²

On **Basic authentication**, the answer is simple: treat it as legacy only. Microsoft states that Basic/web-service access keys are **deprecated and unsupported in Business Central online** starting October 2022, and OAuth2 is recommended instead. Even where older auth types still exist on-premises, they apply to SOAP/OData data access rather than `.app` deployment, so they are not the right foundation for new deployment automation. ¹³

On **security posture**, the clearest recommendation is to prefer **certificates or client assertions** over long-lived client secrets where possible. Microsoft's admin-center docs explicitly say client secrets are shown only for sample purposes and are **not recommended for production**, and they point to X.509 certificates as the recommended pattern. In practice, for a local PowerShell solution, the secure hierarchy is: certificate/client assertion first, short-lived client secret second, interactive login last for ad hoc work only. Store secrets outside source control, ideally in Windows SecretManagement, Windows Credential Manager, or a local secure vault. ¹⁴

Microsoft's broader product behavior also matters for deployment strategy. **DEV extensions are sandbox-only**, and extensions published from Visual Studio Code to online sandboxes can be removed when the sandbox is updated or relocated. PTE uniqueness is enforced by app identity and version rules, so upgrades require keeping the same App ID and increasing version numbers. These characteristics are why the right long-lived production automation target is **PTE upload/install**, not DEV publishing. ¹⁵

Comparison table of approaches

Method	Copyable from GitHub?	PowerShell-friendly?	Supports SaaS / on-prem	Auth method	Pros
Automation API direct REST	Not GitHub-dependent; official docs are sufficient	Yes	SaaS only for PTE upload/install	Entra OAuth client credentials with <code>Automation.ReadWrite.All</code>	Most off pipeline independent; easy to run locally; PowerShell explicit polling for updates
BcContainerHelper <code>Publish-PerTenantExtensionApps</code>	Yes	Yes	SaaS	Entra OAuth via <code>New-BcAuthContext</code> ; client secret or client assertion/device flows	Fastest to set up; a usable PowerShell script; SaaS dependent; polls status; battle-tested; community; Microsoft tooling
Admin Center API app management	Yes , via Microsoft BCTech samples	Yes	SaaS	Entra OAuth client credentials with <code>AdminCenter.ReadWrite.All</code> ; app must be authorized in Admin Center	Best for environments; no lookup, apps, available; updates; update/uninstall; app-operation; tracking
Business Central Administration Shell cmdlets	Official docs rather than GitHub	Yes	On-prem	Local admin / server access; PowerShell remoting from developer PC	Most reliable; official command route; has publish/uninstall; no pipeline changes

Method	Copyable from GitHub?	PowerShell-friendly?	Supports SaaS / on-prem	Auth method	Pros
AL-Go deployment scripts adapted locally	Yes	Yes	SaaS and sandbox/ dev flows; indirectly useful for on-prem design	Depends on target flow; usually <code>New-BcAuthContext</code> plus BC APIs	Excellent reference implementation shows how Microsoft scope selection depends on checking deployment selection
VS Code / AL Language / <code>al_publish</code>	Not GitHub-centric	Limited for unattended PowerShell	Cloud sandbox, cloud production, on-prem per docs; DEV scope constraints still apply	Interactive browser-based MSAL for cloud; Windows auth on some on-prem cases	Good for development publishing; troubleshooting very common in-editor
ALOps External Deployer	Yes	Yes	Primarily on-prem	Depends on how the external deployer service is configured	Useful if you want API on-prem deployment without running shell cmd
SOAP / OData / standard REST data APIs	Not relevant for <code>.app</code> deployment	Technically callable, but not suitable	Data access on SaaS/on-prem	OAuth preferred; legacy Basic only in older/on-prem corners	Good for business service integration

Recommended implementation patterns

Recommended pattern for SaaS

For a controlled, local, pipeline-free SaaS deployment process, I recommend this order:

First choice: **pure PowerShell against the official Automation API** if you want the most future-proof, least opinionated implementation. This is the cleanest option when you can live with explicitly ordering `.app` files by dependency. Microsoft's documented upload flow supports scheduling and schema sync, and it is the official general-purpose SaaS PTE deployment surface reviewed here. ²³

Second choice: **a local wrapper around BcContainerHelper's Publish-PerTenantExtensionApps** if your priority is time-to-value and dependency handling. This is the practical "copy and adapt" option, because the function already does dependency sorting, token renewal, upload streaming, and deployment-status polling. The trade-off is the announced support wind-down in Microsoft's AL-Go direction after October 2027. ²⁴

Use the **Admin Center API** alongside either option when you want to validate the target environment, inspect installed apps, look for required updates, or install/update already-known apps by app ID. Treat direct Admin Center API PTE upload as an emerging option unless your tenant documentation clearly exposes it. ²⁵

Pure PowerShell SaaS script using the official Automation API

The script below implements the documented Microsoft flow. It assumes the `.app` files are already supplied in **dependency order**. If you need automatic dependency sorting, use the helper-based variant that follows. The schedule and schema-sync values match Microsoft's documented `extensionUpload` contract. ⁹

```
param(
    [Parameter(Mandatory = $true)] [string] $TenantId,
    [Parameter(Mandatory = $true)] [string] $ClientId,
    [Parameter(Mandatory = $true)] [securestring] $ClientSecret,
    [Parameter(Mandatory = $true)] [string] $EnvironmentName,
    [Parameter(Mandatory = $false)] [string] $CompanyName,
    [Parameter(Mandatory = $true)] [string[]] $AppFiles,
    [ValidateSet('Current version', 'Next minor version', 'Next major version')]
    [string] $Schedule = 'Current version',
    [ValidateSet('Add', 'Force Sync')]
    [string] $SchemaSyncMode = 'Add',
    [int] $PollSeconds = 10,
    [int] $TimeoutMinutes = 30
)

function Get-PlainText([securestring] $Secret) {
    $bstr = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($Secret)
    try { [Runtime.InteropServices.Marshal]::PtrToStringAuto($bstr) }
    finally { [Runtime.InteropServices.Marshal]::ZeroFreeBSTR($bstr) }
}

function Get-BcAccessToken {
    param(
```

```

        [string] $TenantId,
        [string] $ClientId,
        [securestring] $ClientSecret
    )

    $body = @{
        grant_type    = 'client_credentials'
        client_id     = $ClientId
        client_secret = Get-PlainText $ClientSecret
        scope         = 'https://api.businesscentral.dynamics.com/.default'
    }

    $token = Invoke-RestMethod `
        -Method Post `
        -Uri "https://login.microsoftonline.com/$TenantId/oauth2/v2.0/token" `
        -ContentType 'application/x-www-form-urlencoded' `
        -Body $body

    return $token.access_token
}

function Get-CompanyId {
    param(
        [string] $BaseAutomationUrl,
        [hashtable] $Headers,
        [string] $CompanyName
    )

    $companies = (Invoke-RestMethod -Method Get -Uri "$BaseAutomationUrl/
companies" -Headers $Headers).value
    if (-not $companies) {
        throw "No companies were returned from the target environment."
    }

    if ([string]::IsNullOrEmpty($CompanyName)) {
        return $companies[0].id
    }

    $match = $companies | Where-Object { $_.name -eq $CompanyName } | Select-
Object -First 1
    if (-not $match) {
        throw "Company '$CompanyName' was not found in environment
'$EnvironmentName'."
    }

    return $match.id
}

```

```

function Wait-DeploymentCompletion {
    param(
        [string] $BaseAutomationUrl,
        [string] $CompanyId,
        [hashtable] $Headers,
        [datetime] $StartedAfter,
        [int] $PollSeconds,
        [int] $TimeoutMinutes
    )

    $deadline = (Get-Date).AddMinutes($TimeoutMinutes)

    do {
        Start-Sleep -Seconds $PollSeconds

        $statuses = (Invoke-RestMethod `
            -Method Get `
            -Uri "$BaseAutomationUrl/companies($CompanyId)/
extensionDeploymentStatus" `
            -Headers $Headers).value

        $current = $statuses |
            Where-Object {
                $_.startedOn -and ([datetime]$_.startedOn) -ge $StartedAfter
            } |
            Sort-Object { [datetime]$_.startedOn } -Descending |
            Select-Object -First 1

        if ($current) {
            Write-Host ("Status: {0} | App: {1} {2}" -f $current.status,
                $current.name, $current.appVersion)

            switch ($current.status) {
                'Completed' { return $current }
                'Failed'    { throw "Deployment failed for '$($current.name)'
version '$($current.appVersion)'." }
                'Unknown'  { throw "Deployment returned status Unknown." }
            }
        }
    } while ((Get-Date) -lt $deadline)

    throw "Timed out waiting for extension deployment to finish."
}

$accessToken = Get-BcAccessToken -TenantId $TenantId -ClientId $ClientId -
ClientSecret $ClientSecret
$headers = @{ Authorization = "Bearer $accessToken" }
$baseAutomationUrl = "https://api.businesscentral.dynamics.com/"

```

```

v2.0/$EnvironmentName/api/microsoft/automation/v2.0"
$companyId = Get-CompanyId -BaseAutomationUrl $baseAutomationUrl -Headers
$headers -CompanyName $CompanyName

foreach ($appFile in $AppFiles) {
    if (-not (Test-Path $appFile)) {
        throw "App file not found: $appFile"
    }

    Write-Host "Uploading $appFile"

    $uploadRecord = Invoke-RestMethod `
        -Method Post `
        -Uri "$baseAutomationUrl/companies($companyId)/extensionUpload" `
        -Headers ($headers + @{ 'Content-Type' = 'application/json' }) `
        -Body (@{
            schedule = $Schedule
            schemaSyncMode = $SchemaSyncMode
        } | ConvertTo-Json -Compress)

    if (-not $uploadRecord.systemId) {
        throw "Failed to create extensionUpload record for $appFile"
    }

    $fileStream = [System.IO.File]::OpenRead((Resolve-Path $appFile))
    try {
        Invoke-RestMethod `
            -Method Patch `
            -Uri "$baseAutomationUrl/companies($companyId)/extensionUpload($
($uploadRecord.systemId))/extensionContent" `
            -Headers ($headers + @{
                'Content-Type' = 'application/octet-stream'
                'If-Match' = '*'
            }) `
            -Body $fileStream | Out-Null
    }
    finally {
        $fileStream.Dispose()
    }

    $startedAfter = Get-Date

    Invoke-RestMethod `
        -Method Post `
        -Uri "$baseAutomationUrl/companies($companyId)/extensionUpload($
($uploadRecord.systemId))/Microsoft.NAV.upload" `
        -Headers ($headers + @{ 'If-Match' = '*' }) | Out-Null

```

```

$result = Wait-DeploymentCompletion `
    -BaseAutomationUrl $baseAutomationUrl `
    -CompanyId $companyId `
    -Headers $headers `
    -StartedAfter $startedAfter `
    -PollSeconds $PollSeconds `
    -TimeoutMinutes $TimeoutMinutes

Write-Host ("Completed: {0} {1}" -f $result.name, $result.appVersion)
}

```

This script intentionally leaves **dependency ordering** in your control. If your solution has multiple internal dependencies, either supply `$AppFiles` already in topological order or borrow `BcContainerHelper`'s dependency-sorting logic rather than re-inventing it. Microsoft's own helper function explicitly states that it sorts the apps by dependency before publish/install. ²⁶

Faster SaaS variant using `BcContainerHelper` as the wrapper

If you want a shorter script and you are comfortable with the helper dependency, this is the most pragmatic adaptation path. `BcContainerHelper`'s auth context supports client-secret, client-assertion, refresh-token, password, and device-code-oriented flows, and `Publish-PerTenantExtensionApps` is already designed for the exact problem of pushing PTEs to an online environment. ²⁷

```

Install-Module BcContainerHelper -Scope CurrentUser -Force
Import-Module BcContainerHelper

$authContext = New-BcAuthContext `
    -tenantId $TenantId `
    -clientId $ClientId `
    -clientSecret $ClientSecret `
    -scopes "https://api.businesscentral.dynamics.com/.default"

$appFiles = Get-ChildItem -Path ".\drop" -Filter *.app | Select-Object -
ExpandProperty FullName

Publish-PerTenantExtensionApps `
    -bcAuthContext $authContext `
    -environment $EnvironmentName `
    -companyName $CompanyName `
    -appFiles $appFiles `
    -schemaSyncMode Add `
    -unpublishPreviousVersions

```

Recommended pattern for on-premises

For on-premises, use the **Business Central Administration Shell** cmdlets and invoke them locally on the BC server or remotely from your developer PC over PowerShell remoting. That is the official tooling model Microsoft documents, and it maps cleanly to the publish/sync/install/upgrade lifecycle. ²⁸

A practical flow is:

1. Copy the `.app` file from the developer PC to a temporary folder on the server.
2. `Publish-NAVApp`
3. `Sync-NAVApp`
4. `Install-NAVApp`
5. If the app contains upgrade code, `Start-NAVAppDataUpgrade`
6. Verify with `Get-NAVAppInfo` ²⁹

On-prem PowerShell remoting example

Because Business Central module names vary by release, the script below discovers the newest available BC admin module on the server, then runs the extension lifecycle remotely. The app name/publisher/version parameters are explicit so the sync/install stages remain predictable across BC versions. Microsoft's Admin Shell docs confirm remoting support from a local machine. ³⁰

```
param(
    [Parameter(Mandatory = $true)] [string] $BcServerHost,
    [Parameter(Mandatory = $true)] [string] $ServerInstance,
    [Parameter(Mandatory = $true)] [string] $Tenant = 'default',
    [Parameter(Mandatory = $true)] [string] $LocalAppPath,
    [Parameter(Mandatory = $true)] [string] $AppName,
    [Parameter(Mandatory = $true)] [string] $AppPublisher,
    [Parameter(Mandatory = $true)] [string] $AppVersion,
    [Parameter(Mandatory = $true)] [pscredential] $RemoteCredential,
    [ValidateSet('Add', 'ForceSync', 'Clean', 'Development')]
    [string] $SyncMode = 'Add'
)

$session = New-PSSession -ComputerName $BcServerHost -Credential
$RemoteCredential
try {
    $remoteFolder = "C:\Temp\BCDeploy"
    Invoke-Command -Session $session -ScriptBlock {
        param($remoteFolder)
        New-Item -Path $remoteFolder -ItemType Directory -Force | Out-Null
    } -ArgumentList $remoteFolder

    $remoteAppPath = Join-Path $remoteFolder (Split-Path $LocalAppPath -Leaf)
    Copy-Item -Path $LocalAppPath -Destination $remoteAppPath -ToSession
    $session -Force
```

```

Invoke-Command -Session $session -ScriptBlock {
    param($ServerInstance, $Tenant, $RemoteAppPath, $AppName,
$AppPublisher, $AppVersion, $SyncMode)

    $module = Get-Module -ListAvailable
Microsoft.BusinessCentral.Apps.Management, Microsoft.Dynamics.Nav.Management |
        Sort-Object Version -Descending |
        Select-Object -First 1

    if (-not $module) {
        throw "Business Central admin module not found on server."
    }

    Import-Module $module.Path -Force

    Publish-NAVApp -ServerInstance $ServerInstance -Path $RemoteAppPath -
SkipVerification

    Sync-NAVApp `
        -ServerInstance $ServerInstance `
        -Tenant $Tenant `
        -Name $AppName `
        -Publisher $AppPublisher `
        -Version $AppVersion `
        -Mode $SyncMode

    Install-NAVApp `
        -ServerInstance $ServerInstance `
        -Tenant $Tenant `
        -Name $AppName `
        -Publisher $AppPublisher `
        -Version $AppVersion

    # If this is an upgrade with upgrade codeunits, uncomment:
    # Start-NAVAppDataUpgrade `
    #     -ServerInstance $ServerInstance `
    #     -Tenant $Tenant `
    #     -Name $AppName `
    #     -Publisher $AppPublisher `
    #     -Version $AppVersion

    Get-NAVAppInfo `
        -ServerInstance $ServerInstance `
        -Tenant $Tenant |
        Where-Object {
            $_.Name -eq $AppName -and
            $_.Publisher -eq $AppPublisher -and

```

```

        $_.Version -eq $AppVersion
    }
    } -ArgumentList $ServerInstance, $Tenant, $remoteAppPath, $AppName,
    $AppPublisher, $AppVersion, $SyncMode
}
finally {
    if ($session) { Remove-PSSession $session }
}

```

For on-premises upgrade handling, Microsoft's upgrade guidance is explicit: use `Sync-NAVApp` plus `Start-NAVAppDataUpgrade` when moving to a new app version that contains schema changes and upgrade code. Use `Get-NAVAppInfo` before and after deployment to detect the currently installed version and decide whether you are doing a first install or an upgrade. ³¹

Environment selection, versioning, dependencies, and common failure handling

For **environment selection** in SaaS, the environment name is part of the URL path for both Automation and Admin Center API calls. If you want a strong preflight step, query environments through the Admin Center API before deploying, or at minimum do a harmless GET such as `/companies` against the target Automation API path to verify access and spelling. ³²

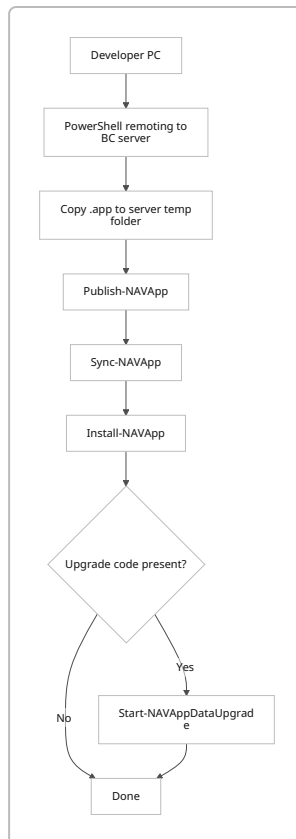
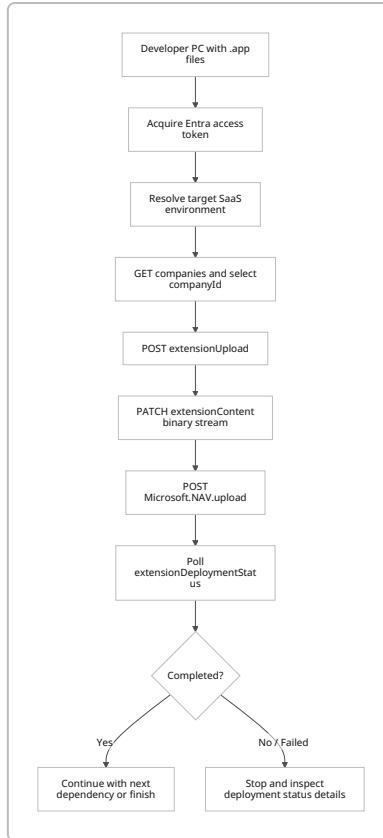
For **versioning**, keep the **same App ID** and **increase version** for upgrades. Microsoft documents that tenant-customization/PTE deployment uses identity and version uniqueness rules, and re-uploading a recompiled package at the same version can fail even if the binary changed. This is especially important when you bypass formal pipelines and deploy manually from local artifacts. ³³

For **dependencies**, the safest rule is to deploy dependency apps first. The official helper script from BcContainerHelper sorts by dependency before publishing, and AL-Go checks/install dependencies before deploying target apps. If you stay pure REST, either pass the files in dependency order or add a manifest parser and topological sort step to your script. ³⁴

For **error handling**, use the platform-native status surfaces. On SaaS Automation API, poll `extensionDeploymentStatus`; on Admin Center API, poll `/operations`; on on-premises, capture cmdlet failures and inspect `Get-NAVAppInfo` and any upgrade status/errors. Microsoft's BCTech sample also shows a useful PowerShell pattern for extracting structured error bodies from failed Admin Center API calls. ³⁵

Deployment flows

The following flow reflects the documented SaaS Automation API sequence and the on-premises admin-shell sequence. ³⁶



Open questions and limitations

The main unresolved area is **direct PTE upload through the Admin Center API**. Microsoft's recent release-plan content says the admin center and its API now support uploading and managing PTEs directly, but the public operational documentation I reviewed still most clearly documents **install/update/uninstall** and app inventory, not the same fully spelled-out upload sequence that exists for the Automation API. Because of that mismatch, I recommend using the **Automation API** today for SaaS upload/install unless your tenant documentation clearly exposes the new admin-center upload endpoints. ³⁷

A second limitation is that **BcContainerHelper** is **highly practical but time-bounded**. It remains very useful and widely copied today, yet Microsoft has announced the AL-Go support phase-out timeline. That does not make it unusable now; it simply means you should treat it as either a short/medium-term wrapper or a source to copy from, not necessarily as a forever dependency in a new internal deployment standard. ³⁸

A third limitation is **version variability** on on-premises. Because no specific Business Central version was supplied, the report uses release-agnostic patterns. In modern on-prem releases, PowerShell 7-compatible `Microsoft.BusinessCentral.*` modules exist, while the older `Microsoft.Dynamics.Nav.*` compatibility path is also still documented. Adjust module import names and exact cmdlet documentation to the installed BC server version. ¹²

In short: for **SaaS**, use **Automation API**, optionally accelerated by **BcContainerHelper**; for **on-prem**, use **Administration Shell cmdlets via PowerShell remoting**; use **Admin Center API** for environment-aware orchestration around those deployments; avoid building new deployment automation on **SOAP/OData/Basic auth** or on purely interactive **VS Code** flows. ³⁹

¹ ⁹ ²³ ³⁶ ³⁹ Introduction to automation APIs - Business Central | Microsoft Learn

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/administration/itpro-introduction-to-automation-apis>

² AL-Go/Actions/Deploy/Deploy.ps1 at main · microsoft/AL-Go · GitHub

<https://github.com/microsoft/AL-Go/blob/main/Actions/Deploy/Deploy.ps1>

³ ¹⁷ ²⁵ Business Central Admin Center API - App Management - Business Central | Microsoft Learn

https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/administration/administration-center-api_app_management

⁴ Publish a Web Service - Business Central | Microsoft Learn

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/webservices/publish-web-service>

⁵ Overview of AL-Go - Business Central | Microsoft Learn

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/al-go/algo-overview>

⁶ ¹⁶ ²⁴ ²⁶ ³⁴ navcontainerhelper/SaaS/Publish-PerTenantExtensionApps.ps1 at main · microsoft/navcontainerhelper · GitHub

<https://github.com/microsoft/navcontainerhelper/blob/master/SaaS/Publish-PerTenantExtensionApps.ps1>

⁷ BCTech/samples/AdminCenterApi at master · microsoft/BCTech · GitHub

https://github.com/microsoft/BCTech/tree/master/samples/AdminCenterApi?utm_source=chatgpt.com

8 **GitHub - HodorNV/ALOps: ALOps · GitHub**

https://github.com/HodorNV/ALOps?utm_source=chatgpt.com

10 **Using Service to Service Authentication - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/administration/automation-apis-using-s2s-authentication>

11 14 **Business Central Administration Center API - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/administration/administration-center-api>

12 18 28 30 **Business Central Administration Shell - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/administration/administration-shell>

13 22 **Web services authentication" - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/webservices/web-services-authentication>

15 **Extension types and scope - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-extension-types-and-scope>

19 **GitHub - microsoft/AL-Go: The plug-and-play DevOps solution for ...**

https://github.com/microsoft/AL-Go?utm_source=chatgpt.com

20 **Publish AL extension to Business Central - al_publish - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/al-agent-tools/al-tool-publish>

21 **GitHub - HodorNV/ALOps-External-Deployer: Enables external deployment in Microsoft Business Central (API Deploy) · GitHub**

<https://github.com/HodorNV/ALOps-External-Deployer>

27 **navcontainerhelper/Auth/New-BcAuthContext.ps1 at main · microsoft/navcontainerhelper · GitHub**

<https://github.com/microsoft/navcontainerhelper/blob/master/Auth/New-BcAuthContext.ps1>

29 **Publish-NAVApp (Microsoft.Dynamics.Nav.Apps.Management) - Dynamics NAV**

https://learn.microsoft.com/en-us/powershell/module/microsoft.dynamics.nav.apps.management/publish-navapp?view=businesscentral-ps-22&utm_source=chatgpt.com

31 **Upgrading Extensions - Business Central | Microsoft Learn**

https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-upgrading-extensions?utm_source=chatgpt.com

32 **Endpoints for the APIs for Microsoft Dynamics 365 Business Central - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/api-reference/v2.0/endpoints-apis-for-dynamics>

33 **Deploying a tenant customization - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/developer/devenv-deploy-tenant-customization>

35 **(automation API) Get extensionDeploymentStatus - Business Central | Microsoft Learn**

https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/administration/api/dynamics_extensiondeploymentstatus_get

37 **Upload, install, and manage PTEs in the admin center**

https://learn.microsoft.com/en-us/dynamics365/release-plan/2026wave1/smb/dynamics365-business-central/upload-install-manage-ptes-admin-center?utm_source=chatgpt.com

38 **Deprecating BcContainerHelper - Business Central | Microsoft Learn**

<https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/al-go/algo-deprecating-bccontainerhelper>